

[2회차] web 과제

발제자: 신영균

기간

- 과제 마감: 7월 16일 23시 59분
- 지각 제출: 7월 17일 23시 59분

Intro

이번 과제는 **협업 과제**입니다. 배정된 팀원과 함께 수행해 주시길 바랍니다

FastAPI 와 **MVC 패턴**을 기반으로 사용자 로그인 기능을 구현하는 과제입니다.

*주의 사항: 팀 과제 레포는 개인 과제 레포와 분리해야 됩니다



MVC 패턴 요약

구성요소	설명
Model	DB와 상호작용, 데이터를 저장 및 처리
View	사용자에게 보여지는 화면 (HTML 등)
Controller	요청을 받아 Model과 View를 연결하고 가공함

Spring 기준

- `Controller` → `Service` → `Repository` 구조
- `DTO` : 계층 간 데이터 전달용 객체

FastAPI에서는

- `Controller` : `user_router.py`
- `Service` : `user_service.py`
- `Repository` : `user_repository.py`
- `DTO` : `user_schema.py` , `base_response.py`

명세

다음 3가지 항목을 구현해야 합니다.

1. `index.html` 꾸미기 (시각적 요소 추가)
2. `user_service.py` 작성
3. `user_router.py` 작성

app 폴더 내의 `##TODO` 와 `index.html` 외에는 수정 금지

`test/` 폴더 수정 금지

API 명세

Base URL: <http://localhost:8000/api/user>

1. Login User

Endpoint : POST `/login`

Request Body

```
{
  "email": "user@example.com",
  "password": "password123"
}
```

Response: 200 OK

```
{
  "status": "success",
  "data": {
    "email": "user@example.com",
    "password": "password123",
    "username": "User123"
  },
}
```

```
"message": "Login Success."
}
```

Errors

Status	메시지
400 이메일을 찾지 못한 경우	"detail": "User not Found."
400 비밀번호가 틀릴 경우	"detail": "Invalid ID/PW"

2. Register User

Endpoint : POST `/register`

Request Body

```
{
  "email": "newuser@example.com",
  "password": "password123",
  "username": "NewUser"
}
```

Response: 201 Created

```
{
  "status": "success",
  "data": {
    "email": "newuser@example.com",
    "password": "password123",
    "username": "NewUser"
  },
  "message": "User registration success."
}
```

Errors

Status	메시지
400 이메일이 이미 존재할 경우	"detail": "User already Exists."

3. Delete User

Endpoint : DELETE /delete

Request Body

```
{
  "email": "user@example.com"
}
```

Response: 200 OK

```
{
  "status": "success",
  "data": {
    "email": "user@example.com",
    "password": "password123",
    "username": "User123"
  },
  "message": "User Deletion Success."
}
```

Errors

Status	메시지
404 이메일을 찾을 수 없는 경우	"detail": "User not Found."

4. Update Password

Endpoint : PUT /update-password

Request Body

```
{
  "email": "user@example.com",
  "new_password": "newpassword123"
}
```

Response: 200 OK

```
{
  "status": "success",
  "data": {
    "email": "user@example.com",
    "password": "newpassword123",
    "username": "User123"
  },
  "message": "User password update success."
}
```

Errors

Status	메시지
404 이메일을 찾을 수 없는 경우	"detail": "User not Found."

참고

- 구현한 함수에 **Docstring** 작성, Typing Annotation 필수
- 에러는 **Service Layer** 에서 발생시켜야 합니다.

1. 실행 경로

- 반드시 `YBIGTA_newbie_team_project/` (**프로젝트 루트 디렉토리**) 경로에서 명령어를 실행해야 합니다. (`app/` 폴더 내부가 아닌, 상위 루트 경로입니다.)

2. 필수 명령어

- 의존성 패키지 설치 (설치 필수):

```
pip install -r requirements.txt
```

- 서버 실행 :

```
uvicorn app.main:app --reload
```

- 테스트 및 검증 실행:

```
# pytest 실행
pytest
# mypy 타입 체크
mypy app/
```

3. 웹 화면(HTML) 접속 주소

서버를 실행한 뒤, 브라우저를 열어 아래 주소로 접속하면 UI 화면을 볼 수 있습니다:

- <http://localhost:8000/static/index.html>

제출 방법

제출 구조:

```
YBIGTA_newbie_team_project/
├─ requirements.txt
├─ app/
│  ├─ __init__.py
│  ├─ main.py
│  ├─ config.py
│  ├─ dependencies.py
│  ├─ static/
│  │  └─ index.html ← 꾸미기
```

```
| | | responses/
| | | | | __init__.py
| | | | | base_response.py
| | | user/
| | | | | __init__.py
| | | | | user_repository.py
| | | | | user_router.py ← 구현
| | | | | user_schema.py
| | | | | user_service.py ← 구현
| | database/
| | | | __init__.py
| | | | users.json
| test/
| | | | __init__.py
| | | | test_user_router.py
| | | | test_user_service.py
| ...
```

채점 기준

- test/test_user_service.py , test/test_user_router.py
 - `pytest` , `mypy` 테스트 모두 통과해야 제출 인정
- `index.html` 을 조금이라도 꾸몄다면 **제출**

테스트 코드를 제공해드렸기 때문에 **모두** 통과해야 제출로 인정

통과하지 못하면 **미흡**, 제출하지 않으면 **미제출**입니다